

Access Control and SSRF

Efren Antonio

ITIS 4221

April 25, 2025

ACCESS CONTROL & SSRF REPORT

TABLE OF CONTENTS

<u>Section</u>	<u>Page #</u>
1.0 General Information	3
1.1 Purpose	3
2.0 SSRF Vulnerability	4
2.1 Exploitation	4
2.2 Code Vulnerability	5
2.3 Fix	6
3.0 Information Disclosure Vulnerability	7
3.1 Exploitation	7
3.2 Code Vulnerability	8
3.3 Fix	8
4.0 Access Control Vulnerability	9
4.1 Exploitation	9
4.2 Code Vulnerability	10
4.3 Fix	10
5.0 Apache Shiro Access Control Configuration	13
5.1 Original File	13
5.2 Modified File	14

1.0 General Information

1.1 Purpose

The purpose of this security assessment is to identify, exploit, and mitigate several critical vulnerabilities within the `penetration_test` application. The vulnerabilities addressed include a Server-Side Request Forgery (SSRF) flaw, an Insecure Direct Object Reference (IDOR) weakness, and a broken access control found under the performance evaluation section. Additionally, this report documents improvements made to the Apache Shiro access control configuration without modifying the original Java codebase. The IDE IntelliJ was used to fix the vulnerabilities, and Burp Suite was utilized to identify and demonstrate each vulnerability. Screenshots capturing exploitation, vulnerable code, fixed code, and verification of successful mitigation are included to document the remediation process.

2.0 SSRF Vulnerability

The SsrfController class in the penetration_test application is vulnerable to Server-Side Request Forgery (SSRF) due to a lack of input validation on externally supplied URL parameters. This flaw is present in the /ssrf/ endpoint that allows user-submitted values to be used in a backend HTTP request to another internal or external service. An attacker can leverage this to make unauthorized requests on behalf of the server, potentially accessing internal systems or metadata services.

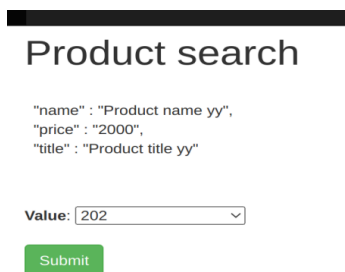
2.1 Exploitation

By intercepting the form submission with Burp Suite, an attacker can modify the program, parameter, and param_value fields to create a malicious request that points to an internal or unauthorized resource. In this example, the attacker changes the program field to 'localhost./sql_injection' and both the parameter and param_value fields to 'test.' This forces the application to access the penetration test home page log in screen.

```
Origin: http://localhost:8081
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://localhost:8081/ssrf/
Accept-Encoding: gzip, deflate, br
Cookie: JSESSIONID=7F260CB80EDCF13281320FA80E873392
Connection: keep-alive

program=item&parameter=id&param_value=202
```

Figure 1: Original Request Sent by the Form



Product search

\"name\" : \"Product name yy\",
\"price\" : \"2000\",
\"title\" : \"Product title yy\"

Value: 202

Submit

Figure 2: Original Response Result

```

Origin: http://localhost:8081
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://localhost:8081/ssrf/
Accept-Encoding: gzip, deflate, br
Cookie: JSESSIONID=7F260CB80EDCF13281320FA80E873392
Connection: keep-alive
...
program=localhost./sql_injection/&parameter=test&param_value=test|

```

Figure 3: Exploited Request Sent by the Form

Product search

Login page

Username:

Password:

[Log In](#)

New user? [Register here](#)

Value:

[Submit](#)

Figure 4: Exploited Response Results

2.2 Code Vulnerability

The code vulnerability is located in the `product_search_post()` method of `SsrfController.java`, the application accepts user input and embeds it directly into a backend URL without validating the destination. There are no restrictions on the values of `program`, `parameter`, or `param_value`, and no filtering or domain validation is in place.

```

try {
    URL obj = new URL(spec: "http://localhost:8081/ssrf/product/" + program + "/" + parameter + "=" + param_value);
    HttpURLConnection con = (HttpURLConnection) obj.openConnection();
    con.setRequestMethod("POST");
}

```

Figure 5: Vulnerable Code

2.3 Fix

To prevent SSRF, the updated code implements strict validation on the program parameter, allowing only 'item' or 'design' as valid options. If the user submits any other value, the request is immediately rejected with an error response, and no connection to external or internal services is made. By enforcing this allowlist, the application prevents attackers from crafting malicious URLs that could lead to unauthorized internal network access or server-side resource fetching, effectively mitigating the SSRF vulnerability.

```
try {
    // Only allow trusted programs
    if (!program.equals("item") && !program.equals("design")) {
        response_data.put("status", "error");
        response_data.put("msg", "Invalid program requested.");
        return response_data;
    }
    URL obj = new URL("http://localhost:8081/ssrf/product/" + program + "/" + parameter + "=" + param_value);
    HttpURLConnection con = (HttpURLConnection) obj.openConnection();
    con.setRequestMethod("POST");
```

Figure 6: Fixed Code

When the attacker tries to change the program field to something other than the valid options, they are met with the following 'invalid program requested' screen:

```
12 Origin: http://localhost:8081
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: http://localhost:8081/ssrf/
17 Accept-Encoding: gzip, deflate, br
18 Cookie: JSESSIONID=7F260CB80EDCF13281320FA80E873392
19 Connection: keep-alive
20
21 program=localhost./sql_injection/&parameter=test&param_value=test
```

Figure 7: Exploited Request Sent After Fix

Product search

Invalid program requested.

Value:

Submit

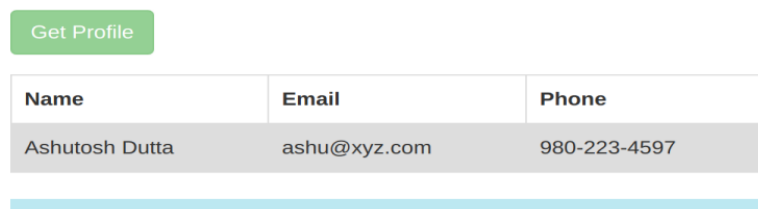
Figure 8: Fixed Exploit Error Screen

3.0 Information Disclosure Vulnerability

The IdorController class in the penetration_test application contains an information disclosure vulnerability within the /idor/own endpoint. After a customer logs into the application, they can retrieve their profile information through this endpoint. However, the server returns the entire customer session object without any filtering, inadvertently exposing sensitive fields such as the user's password and pay amount. This excessive exposure of internal attributes poses a serious security risk, as customers could access backend-specific data not intended for client-side consumption.

3.1 Exploitation

Once logged in, a customer can access the /idor/own endpoint to view their profile information, which includes a few sensitive fields in the response. Using Burp Suite, an attacker can view the internal details that aren't shown in the webpage.



A screenshot of a web application interface. At the top, there is a green button labeled 'Get Profile'. Below the button is a table with three columns: 'Name', 'Email', and 'Phone'. The table contains one row of data for a user named 'Ashutosh Dutta' with email 'ashu@xyz.com' and phone number '980-223-4597'. Below the table is a light blue horizontal bar.

Name	Email	Phone
Ashutosh Dutta	ashu@xyz.com	980-223-4597

Figure 9: Regular Information On Webpage

```
{
  "id": "886359",
  "password": "456",
  "name": "Ashutosh Dutta",
  "phone": "980-223-4597",
  "amount": "$51,000",
  "performance":
    "Job knowledge: S</br>Work quality: F</br>Attendance: E</br>
    Communication: S",
  "email": "ashu@xyz.com"
}
```

Figure 10: Sensitive Data In Response

3.2 Code Vulnerability

The code vulnerability is located in the `idor_profile_own()` method of the `IdorController` class. Instead of selectively returning only non-sensitive fields, the method serializes and sends the entire `logged_in_customer` session object back to the client without sanitization.

```
List list = new ArrayList<>();
String json = "";
try {
    list.add(customer_sessionInfo);
    json = objectMapper.writeValueAsString(list);
}
```

Figure 11: Vulnerable Code

3.3 Fix

To prevent this vulnerability, the `idor_profile_own()` method is updated to sanitize the session object before returning it. The 'password' and 'amount' attributes are removed from the 'session_customerInfo' dictionary. Only the data required to build the necessary table on the webpage remain.

```
List list = new ArrayList<>();
String json = "";
try {
    list.add(customer_sessionInfo);
    customer_sessionInfo.remove(key: "password");
    customer_sessionInfo.remove(key: "amount");
    json = objectMapper.writeValueAsString(list);
}
```

Figure 12: Fixed Code

```
{
  "id": "886459",
  "name": "Moumita Das",
  "phone": "980-126-5874",
  "performance":
    "Job knowledge: F</br>Work quality: S</br>Attendance: E</br>Communication: S",
  "email": "mpurba@xyz.com"
}
```

Figure 13: Only Necessary Data In Response

4.0 Access Control Vulnerability

The IdorController class in the penetration_test application is also vulnerable to an access control vulnerability attack. After logging in as a customer, users can manually change a requested_customer_id value in the URL to retrieve another customer's information without proper authorization. The application fails to enforce that users should only access their own profiles, leading to unauthorized access to sensitive customer details.

4.1 Exploitation

Once logged in as a legitimate user, an attacker can intercept the request to /idor/info?requested_customer_id=<id> and modify the requested_customer_id to another customer's ID using Burp Suite tools. In this example, the attacker is logged in to the 'mpurba@xyz.com' account and is able to access the performance evaluation of the 'ashu@xyz.com' account by intercepting and modifying the customer id to '886359' which corresponds to the 'ashu' account.

Get Profile: There is often data in the raw response that doesn't show up on the screen/page. Please change code accordingly to apply restrictions on viewing all data.

Get Profile		
Name	Email	Phone
Moumita Das	mpurba@xyz.com	980-126-5874

Performance Evaluation: View someone else's result by man in the middle attack. Please change code accordingly to apply restrictions on viewing other's data.

Performance Evaluation		
Name	Email	Performance Evaluation(%)
Moumita Das	mpurba@xyz.com	Job knowledge: F Work quality: S Attendance: E Communication: S

Figure 14: 'mpurba@xyz.com' Customer Profile

Request

	Pretty	Raw	Hex
1	GET /idor/info/?requested_customer_id=886359&_=1745873630704		HTTP/1.1
2	Host: localhost:8081		
3	sec-ch-ua-platform: "Linux"		
4	X-Requested-With: XMLHttpRequest		
5	Accept-Language: en-US,en;q=0.9		

Figure 15: Changed 'requested_customer_id' to '886359'

Get Profile: There is often data in the raw response that doesn't show up on the screen/page. Please change code accordingly to apply restrictions on viewing all data.

Get Profile

Name	Email	Phone
Moumita Das	mpurba@xyz.com	980-126-5874

Performance Evaluation: View someone else's result by man in the middle attack. Please change code accordingly to apply restrictions on viewing other's data.

Performance Evaluation

Name	Email	Performance Evaluation(%)
Ashutosh Dutta	ashu@xyz.com	Job knowledge: S Work quality: F Attendance: E Communication: S

Figure 16: Unauthorized Access to 'ashu@xyz.com' Performance Evaluation by 'mpurba@xyz.com' Account

4.2 Code Vulnerability

In the `idor_performance_info()` method, the server retrieves user profiles based only on the supplied `requested_customer_id`, without verifying if the logged-in user is authorized to view the data. There is no validation that `requested_customer_id` matches the current logged-in user's ID.

```
for (Map.Entry<String, Map<String, String>> entry : customerInfo.entrySet()) {
    String key = entry.getKey();
    Map<String, String> value = entry.getValue();
    String each_customer_id = value.get("id");

    if (requested_customer_id.equals(each_customer_id)) {
        requested_info.put("msg", "You have successfully seen data");
        requested_info.put("name", value.get("name"));
        requested_info.put("email", key);
        requested_info.put("performance", value.get("performance"));
    }
}
return_val = objectMapper.writeValueAsString(requested_info);
```

Figure 17: Vulnerable Code

4.3 Fix

To fix the IDOR vulnerability, a check was added to ensure the `requested_customer_id` matches the ID stored in the logged-in user's session. Only the logged-in user can view their own profile information. Any attempt to access another user's profile is blocked with an error message.

```
try {
    ObjectMapper objectMapper = new ObjectMapper();
    if (session.getAttribute("logged_in_customer") == null) {
        requested_info.put("status", "error");
        requested_info.put("msg", "Please login as customer first");
        return objectMapper.writeValueAsString(requested_info);
    }
    // Get the logged-in customer info
    Map<String, String> session_customerInfo = (Map<String, String>) session.getAttribute("logged_in_customer");
    String logged_in_customer_id = session_customerInfo.get("id");

    // Only allow access if requested ID matches logged-in ID
    if (!requested_customer_id.equals(logged_in_customer_id)) {
        requested_info.put("status", "error");
        requested_info.put("msg", "Access denied: You can only view your own profile");
        return objectMapper.writeValueAsString(requested_info);
    }
}
```

Figure 18: Fixed Code

While logged into the 'mpurba@xyz.com' account, an attempt to access the 'ashu@xyz.com' performance evaluation is met with an 'access denied' error message.

```
1 GET /idor/info/?requested_customer_id=886359&_=1745873630704 HTTP/1.1
2 Host: localhost:8081
3 sec-ch-ua-platform: "Linux"
4 X-Requested-With: XMLHttpRequest
```

Figure 19: Changed 'requested_customer_id' To '886359'

Get Profile: There is often data in the raw response that doesn't show up on the screen/page. Please change code accordingly to apply restrictions on viewing all data.

Get Profile

Name	Email	Phone
Moumita Das	mpurba@xyz.com	980-126-5874

Performance Evaluation: View someone else's result by man in the middle attack. Please change code accordingly to apply restrictions on viewing other's data.

Performance Evaluation

Access denied: You can only view your own profile

Figure 20: Access Denied To View Another User's Performance Evaluation

5.0 Apache Shiro Access Control Configuration

Apache Shiro is a powerful and flexible Java security framework that handles authentication, authorization, and session management. In this penetration test application, the 'shiro.ini' file is used to configure user credentials, define roles, and specify which roles have access to which resources. Instead of modifying any Java code, this section demonstrates how access permissions were changed by directly editing the configuration file.

4.1 Original File

The original Apache Shiro configuration file defines user accounts and roles within two main sections: [users] and [roles].

```
[users]
# user 'root' with password 'secret' and the 'admin' role
root = secret, admin
# user 'guest' with the password 'guest' and the 'guest' role
guest = guest, guest
# user 'presidentskroob' with password '12345' ("That's the same combination on
# my luggage!!!" ;)), and role 'president'
presidentskroob = 12345, president
# user 'darkhelmet' with password 'ludicrousspeed' and roles 'darklord' and 'schwartz'
darkhelmet = ludicrousspeed, darklord, schwartz
# user 'lonestarr' with password 'vespa' and roles 'goodguy' and 'schwartz'
lonestarr = vespa, goodguy, schwartz

# -----
# Roles with assigned permissions
#
# Each line conforms to the format defined in the
# org.apache.shiro.realm.text.TextConfigurationRealm#setRoleDefinitions JavaDoc
# -----
[roles]
# 'admin' role has all permissions, indicated by the wildcard '*'
admin = *
# The 'schwartz' role can do anything (*) with any lightsaber:
schwartz = lightsaber:*
# The 'goodguy' role is allowed to 'drive' (action) the winnebago (type) with
# license plate 'eagle5' (instance specific id)
goodguy = winnebago:drive:eagle5
```

Figure 21: Original 'shiro.ini' File

```
INFO [org.apache.shiro.session.mgt.AbstractValidatingSessionManager] - Enabling session validation scheduler
INFO [Quickstart] - Retrieved the correct value! [aValue]
INFO [Quickstart] - There is no user with username of billchu
INFO [Quickstart] - User [null] logged in successfully.
INFO [Quickstart] - Hello, mere mortal.
INFO [Quickstart] - Sorry, lightsaber rings are for schwartz masters only.
INFO [Quickstart] - Sorry, you aren't allowed to drive the 'eagle5' winnebago!
INFO [Quickstart] - Sorry, you aren't allowed to drive the 'tesla5' tesla!
```

Figure 22: Original File Running

4.2 Modified File

The modified Apache Shiro configuration updates the access control to create a new role '49sd', assigns it to the user 'billchu', and grants the new role permissions to drive all Teslas and all Winnebagos. After applying the modified configuration and running the application, the user billchu is now able to drive any Tesla and any Winnebago. No changes were made to the Java program; only the shiro.ini file was updated.

```
[users]
# user 'root' with password 'secret' and the 'admin' role
root = secret, admin
# user 'guest' with the password 'guest' and the 'guest' role
guest = guest, guest
# user 'presidentskroob' with password '12345' ("That's the same combination on
# my luggage!!!" ;)), and role 'president'
presidentskroob = 12345, president
# user 'darkhelmet' with password 'ludicrousspeed' and roles 'darklord' and 'schwartz'
darkhelmet = ludicrousspeed, darklord, schwartz
# user 'lonestarr' with password 'vespa' and roles 'goodguy' and 'schwartz'
lonestarr = vespa, goodguy, schwartz
# user 'billchu' with password '123' and '49sd' role
billchu = chu, 49sd

# -----
# Roles with assigned permissions
#
# Each line conforms to the format defined in the
# org.apache.shiro.realm.text.TextConfigurationRealm#setRoleDefinitions JavaDoc
# -----
[roles]
# 'admin' role has all permissions, indicated by the wildcard '*'
admin = *
# The 'schwartz' role can do anything (*) with any lightsaber:
schwartz = lightsaber:*
# The 'goodguy' role is allowed to 'drive' (action) the winnebago (type) with
# license plate 'eagle5' (instance specific id)
goodguy = winnebago:drive:eagle5
# The '49sd' role is allowed to 'drive' any tesla and any winnebago
49sd = tesla:drive:*, winnebago:drive:*
```

Figure : Modified 'shiro.ini' File

```
[Quickstart] - Retrieved the correct value! [aValue]
[Quickstart] - User [billchu] logged in successfully.
[Quickstart] - Hello, mere mortal.
[Quickstart] - Sorry, lightsaber rings are for schwartz masters only.
[Quickstart] - You are permitted to 'drive' the winnebago with license plate (id) 'eagle5'.
[Quickstart] - You are permitted to 'drive' the tesla with license plate (id) 'tesla5'. Here
```

Figure: Modified Permissions For 'billchu'